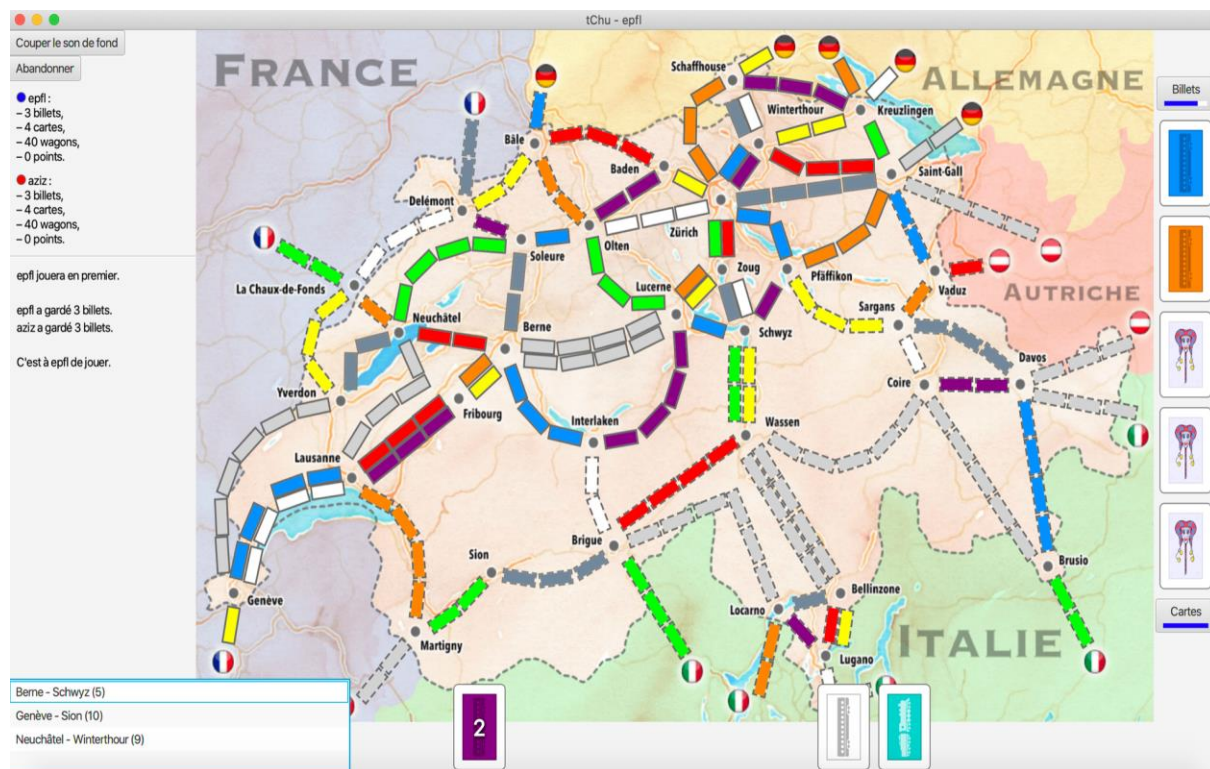


Rapport des améliorations ajoutées

Représenté par Ahmed Aziz BEN HAJ HMIDA (SCIPER: 310934) et Aziz LAADHAR (SCIPER: 315196) :



Introduction :

Dans cette partie bonus, on a amélioré l'affichage de l'interface graphique de notre jeu. On a aussi ajouté des effets sonores au jeu, un type de cartes supplémentaire ainsi qu'une animation sur les routes obtenues et deux actions, notamment rejouer et abandonner.

Changements importants avant de lancer le jeu :

Le jeu ne va pas être correctement exécuté au début : il faut ajouter la commande `javafx.controls`, `javafx.media` dans Run puis Edit Configurations ainsi qu'il faut, pour les instances media ajoutées, (3 déclarées dans `GraphicalPlayerAdapter` (lignes 54,56 et 58) et 1 dans

GraphicalPlayer (ligne 87)) changer les chemins des fichiers .mp3 par celui des emplacements de ces mêmes fichiers dans votre ordinateur.

Les améliorations apportées au jeu sont :

1) On a fait quelques changements dans les feuilles CSS, notamment map.css, colors.css et decks.css, on a amélioré les couleurs utilisées et la grandeur des rectangles des wagons pour une meilleure clarté du jeu et on a choisi la couleur des cartes JOKER ajoutées (décrites plus bas) ainsi que l'image du font de ces cartes.

2) On a ajouté la possibilité de rejouer, en effet, à la fin du jeu une fenêtre graphique apparaît dans laquelle il y'a possibilité de rejouer ou fermer, si les deux joueurs choisissent l'option rejouer le jeu recommence de nouveau sinon, dans le reste des cas, le jeu se termine et on a procédé comme ceci:

- classe GameState : on a créé le type énuméré PlayerRestartResponse et un attribut ALL ce type énumère nécessaire à la sérialisation.

- interface Player : on a ajouté une méthode abstraite qui retourne l'un des types de PlayerRestartResponse et qui prend comme argument le texte des scores et du fin du jeu.

- classe GraphicalPlayer : on a ajouté une méthode gameover() qui crée la fenêtre graphique et on a ajouté que le fait d'appuyer sur le bouton rouge d' « exit » cela signifie que la valeur DOESNT_WANT_TO_REPLAY du type énuméré PlayerRestartResponse soit choisie en appelant la méthode onRequestReplay() de l'interface ReplayHandler ajoutée dans ActionHandlers et on a créé, comme on a fait avant, deux boutons « rejouer » et « fermer » de la fenêtre graphique et, pour chaque bouton, on a fait l'appel à la méthode onRequestReplay() de l'interface ReplayHandler avec comme argument les deux valeurs WANTS_TO_REPLAY et DOESNT_WANT_TO_REPLAY de PlayerRestartResponse.

- classe GraphicalPlayerAdapter, on a implémenté la méthode gameOver() de Player ou on a défini un ReplayHandler, comme dans le cas des tickets et cartes, qui prend un playerId et un playerRestartResponse et retourne la décision du joueur, et pour que le choix des deux joueurs se fait en parallèle, dans la classe Game, on a déclaré deux threads et un queue(file) threadReplay avec une capacité de deux et si les deux joueurs choisissent de rejouer on appelle de nouveau (Game.play(players, playerNames, tickets, rng)).

- classe RemotePlayerClient : dans la méthode run() et dans le switch on a ajouté le cas de GAMEOVER (type ajouté dans l'énum MessageId) qui sérialise la valeur retournée avec l'attribut PLAYERRESTARTRESPONSE ajouté dans Serdes.

- classe RemotePlayerProxy : on a ajouté la méthode gameOver() qui envoie au client afin de recevoir son choix et retourne le choix du joueur en le dé-sérialisant.

3) On a ajouté aussi des **effets sonores** lors des tirages des tickets, cartes, l'obtention d'une route et le début du tour d'un joueur (le joueur qui entend le son va se rendre compte que c'est son tour qui commence) ainsi qu'un font de music du jeu et un bouton qui coupe et remet le son et on a procédé comme ceci:

- interface BackgroundSoundStateHandler dans ActionHandlers : ajoutée avec sa méthode onToggleSoundState() responsable de la coupure et l'activation du son.

- classe GraphicalPlayerAdapter : on a déclaré 3 attributs de type MediaPlayer avec des noms assez explicatifs et dans le constructeur de cette classe on a créé des instances de ces attributs de MediaPlayer avec comme argument les fichiers .mp3 pour chaque type (dont l'emplacement doit être mis à jour lorsque le code s'exécute sur un nouvel ordinateur) et après pour chaque action on a joué le son correspondant (.play()).

- classe GraphicalPlayer : on a déclaré un attribut MediaPlayer pour le son du font du jeu (et on a créé l'objet correspondant dans une runLater() pour que le son se joue, même si l'interface graphique n'est pas la fenêtre principale de l'écran (pour le reste des détails du code, ils sont mentionnés dans des commentaires au-dessus du code correspondant)) et pour la méthode createInfoView() (c'est où on a créé le bouton « Couper le son de fond ») on a ajouté un argument de type ActionHandlers.BackgroundSoundStateHandler qui gère la coupure et l'activation du son (on n'a pas trouvé de méthode qui indique si le son est en train d'être joué ou non alors on a travaillé avec la méthode trouvée isMute()).

4) On a aussi **ajouté un bouton « abandonner »** sur lequel le joueur ne peut pas appuyer que si c'est à lui de jouer et il veut quitter la partie avant sa fin en se déclarant comme perdant et on a procédé comme ceci:

- interface SurrenderHandler dans ActionHandlers : ajoutée avec sa méthode onSurrender() responsable de l'action « abandonner » et on a déclaré SURRENDER comme action de TurnKind dans Player.

- classe GraphicalPlayer : on a ajouté un argument de type SurrenderHandler dans la méthode startTurn() et on a dans la méthode de même nom dans GraphicalPlayerAdapter définit le SurrenderHandler et l'ajouté comme argument à l'appel de nextTurn() et on a ajouté SurrenderHandler comme argument à createInfoView().

- classe Game : on a ajouté le cas de SURRENDER dans la switch et on a changé la condition du fin du jeu dans la boucle while.

5) On a aussi **ajouté une indication sur la route qui vient d'être obtenue** en animant la route pendant 2 secondes afin que le deuxième joueur se rend compte de quelle route l'autre joueur s'est emparé et on a procédé comme dans une vidéo qu'on a trouvé sur les animations JavaFx et on a tout simplement changé la durée de l'animation à deux secondes avant de l'arrêter en utilisant un thread (le reste des détails sont au niveau du code).

6) On a aussi **ajouté huit cartes JOKER** pour que le nombre total des cartes devienne 118, on a aussi ajouté que le joueur ne peut qu'utiliser une carte JOKER à la fois, qui remplace n'importe quelle carte couleur afin de s'emparer d'une route OVERGROUND, seulement, et on a procédé comme ceci:

- classe Route : on a changé la méthode possibleClaimCards().
- classes CARD , COLOR et les feuilles CSS : quelques changements.

Remarque :

On a oublié pour le dernier rendu d'utiliser la constante LONGEST_TRAIL_BONUS_POINTS qu'on l'a utilisé dans cette partie bonus ainsi que on a déclaré de nouvelles constantes comme JOKER_CARDS_COUNT, changé quelques constantes comme TOTAL_CARDS_COUNT (dans la classe Constants) et ajouté aussi un attribut SURRENDER (dans StringsFr).